Docker 部署数据库

Redis · MySQL · MongoDB

目标

数据库是有状态服务, 三件标配要记牢

- 数据库特性: 删容器数据就没 → 必须**数据卷**
- 三件标配: `-v 数据目录` + `-e 账号密码` + `-p 端口`
- Redis(重点): redis-cli 进终端验证
- MySQL(重点): root 密码 + mysql 客户端

一、数据库的特殊性

> 别把数据和容器绑在一起

前面删了能重建, 数据库删了数据全没

- Web、语言应用: 可写层丢了, 重建即可
- 数据库: 数据就在可写层里 **随容器销毁而消失**

> 所以数据库部署 = 基础模板 + 数据卷持久化

三件标配, 一个模板填空

```
docker run -d --name [名] \
  -p 宿主端口:容器端口 \      # 让外面能连
  -e 环境变量 \                # 账号/密码/初始库
  -v 数据卷:/数据目录 \        # 数据别随容器丢
  [官方镜像]
```



> 三库都一样, 只是换镜像、端口、数据目录、客户端

二、Redis (重点)

> 最轻的有状态服务

默认跑起来, exec 进终端就能连

```
docker run -d --name my-redis -p 6379:6379 redis:7
docker exec -it my-redis redis-cli ping    # PONG
```



端口6379 · 数据目录 /data · 客户端 redis-cli

挂数据卷 + 设密码, 才算"像样"

```
docker run -d --name my-redis \
  -p 6379:6379 \
  -v redisdata:/data \
  redis:7 --requirepass mypass --appendonly yes

docker exec -it my-redis redis-cli -a mypass
> SET city beijing
> GET city      # "beijing"
```



删容器再启, 数据靠卷活着

```
docker rm -f my-redis
docker run -d --name my-redis -p 6379:6379 \
  -v redisdata:/data redis:7 --requirepass mypass --appendonly yes
docker exec -it my-redis redis-cli -a mypass GET city  # 还是 beijing
```



验证"删容器, 数据不丢"

三、MySQL(重点)

> 密码写进环境变量

必配 MYSQL_ROOT_PASSWORD, 否则起不来

```
docker run -d --name my-mysql \
  -p 3306:3306 \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -e MYSQL_DATABASE=demo \            # 可选: 自动建库
  -v mysql-data:/var/lib/mysql \
  mysql:8
```



端口3306 · 数据目录 /var/lib/mysql

exec 进终端, 建库建表写读

```
docker exec -it my-mysql mysql -uroot -proot123
mysql> CREATE DATABASE shop;
mysql> USE shop;
mysql> CREATE TABLE users(id INT PRIMARY KEY, name VARCHAR(20));
mysql> INSERT INTO users VALUES (1, 'tom');
mysql> SELECT * FROM users;
```



> 一律 `-e MYSQL_ROOT_PASSWORD` 起, 用 mysql 客户端连

常见报错排查

现象	处理
容器反复重启	没设 root 密码 / 旧数据不匹配 → `docker logs`
Access denied	确认 `-e` 密码与连接一致
端口占用	换宿主端口, 如 `-p 3307:3306`

四、MongoDB (了解)

> 同套路, 换词

mongo 镜像 + mongosh 客户端

```
docker run -d --name my-mongo \
  -p 27017:27017 \
  -v mongodata:/data/db \
  mongo:7

docker exec -it my-mongo mongosh
> db.site.insertOne({name:"docker"})
> db.site.find()
```



端口27017 · 数据目录 /data/db · 客户端 mongosh

三库速览, 一个套路

库	镜像	端口	数据目录	客户端
Redis	redis	6379	/data	redis-cli
MySQL	mysql	3306	/var/lib/mysql	mysql
MongoDB	mongo	27017	/data/db	mongosh

> 记住"端口 + 数据目录 + 客户端", 三库通用

五、实战一

部署并连接 Redis (作业)

> 进入容器终端, 读写验证

获取镜像, 起带持久化的 Redis

```
docker pull redis:7
docker run -d --name my-redis -p 6379:6379 \
  -v redisdata:/data \
  redis:7 --requirepass mypass --appendonly yes

docker exec -it my-redis redis-cli -a mypass
> PING           # PONG
> SET city beijing
> GET city       # "beijing"
> exit
```



六、实战二

部署并连接 MySQL

> 建库建表, 数据落卷

建一张表，写完删容器仍能读到

```
docker run -d --name my-mysql -p 3306:3306 \
  -e MYSQL_ROOT_PASSWORD=root123 -v mysql-data:/var/lib/mysql mysql:8

docker exec -it my-mysql mysql -uroot -proot123
mysql> CREATE DATABASE shop; USE shop;
mysql> CREATE TABLE users(id INT PRIMARY KEY, name VARCHAR(20));
mysql> INSERT INTO users VALUES (1,'tom'); SELECT * FROM users;

# 删容器→重建→数据还在
docker run -d --name my-mysql -p 3306:3306 \
  -e MYSQL_ROOT_PASSWORD=root123 -v mysql-data:/var/lib/mysql mysql:8
docker exec -it my-mysql mysql -uroot -proot123 \
  -e "USE shop; SELECT * FROM users;"     # tom 还在
```



一张表带走全部要点

方面	要点
数据库特性	有状态 → 删容器数据没了 → **必须数据卷**
三件标配	`-v 数据目录` + `-e 账号密码` + `-p 端口`
Redis	6379 · /data · redis-cli · --requirepass
MySQL	3306 · /var/lib/mysql · MYSQL_ROOT_PASSWORD
MongoDB	27017 · /data/db · mongosh
进终端	`docker exec -it 容器 客户端` 三库统一
排查	起不来先 `docker logs`

数据卷是数据库的"保险柜", 容器只是临时来拿货的

下一步: 两讲综合案例 —— 先用 Docker 做静态网站测试, 再做 Java Web 应用上 Tomcat, 把全部所学串起来

谢谢大家

Q & A